RAFT in Metamorph

I have been looking at the thing where every other webapps like chat-gpt and gemini and claude adapt to improve their apps experience.
Where I kept everything to comfortably run on local system, so we can use either a Hybrid RAG system or a RAFT system.

RAFT is basically Retrieval Augmented Fine Tuning, which helps that basic llm model to be the more domain specific model. Means the model will be trained on that specific domain and then have a RAG system to improve its capabilities in a more advanced way so that it can better perform in compare to the base LLM (not fine-tuned).

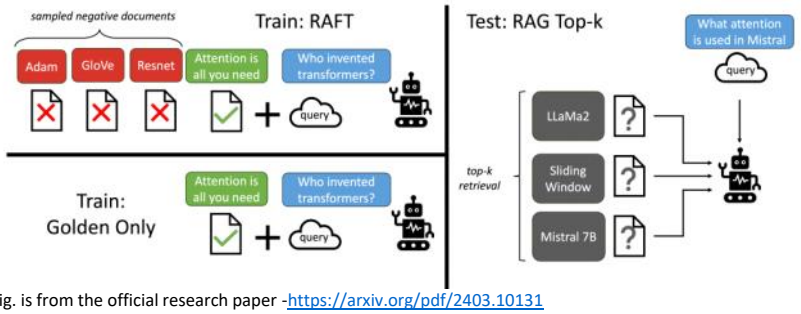


*fig. is from the official research paper -https://arxiv.org/pdf/2403.10131

Ex.- So from the image we can clarify that the model behaves like the one student of the class that has more knowledge on that specific field and provided an open book which make it even more smart than rest of all.

To fine-tune a LLm locally might not be possible, cause the main bottlenecking condition will be the VRAM.
I have used a gemma4e4b which I quantization to 4bit to fit in the vram perfectly.
To quantize a LLM model by yourself,

- 1st of all make sure that your system hardware can perfectly able to quantize that model, for example, if you are using a gemma4e4b model of 8B parameter which weights 16gb, to quantize that to 4bit (q4_K_M) it will consume about 6gb of VRAM and for 8bit, it will consume about roughly 8gb of VRAM .
- Then make sure that the llama.cpp is installed, else go for this official GitHub : https://github.com/ggml-org/llama.cpp/releases. From that make sure you system is mentioned.
- Then follow this line to quantize a model :
    - For windows: llama-quantize.exe <model.gguf> <model_new_name.gguf> <quantization Form>(q4_K_M or q5_K_S ...).
    - For MAC : llama-quantize <model.gguf> <model_new_name.gguf> <quantization Form>(q4_K_M or q5_K_S ...).

After this you are good to go for fine-tuning.
To fine-tune make sure you got the dataset upon which you are going to fine-tune is cleaned properly with not much outliers.
You can consider the dataset from the Kaggle or something like it. I will go with Kaggle cause I'm more familiar with it.
After you got the dataset make sure that your system hardware can fine-tune the model further cause, fine-tuning a model recures heavy resource and efforts. Else you can consider google collab which gives upto 15gb of free VRAM access set the runtime type to T4 GPU, which can help in fine-tune a model and then apply fine-tune methods like LoRA or QLoRA.
And then train the model and export it on .bin or .safetensors weight files.
To fine-tune a model on collab better to pull the existing model or a quantized model from Hugging Face and then fine-tune it.

*before that make this clear that you can't fine-tune a model that is quantized on local and the weights are saved in a frozen (the mathematical calculation are being erased to fine-tune it further) i.e. in .gguf or .awq form mainly we save it as a .gguf file to run things locally.

*we can fine-tune a model from huggingface cause the quantized model is because of PEFT (parametric efficient fine tune) paired with bitesandbytes (bnb) file format.

*PEFT - it do add a layer of knowledge over the base model by freezing the base model and trains only a small set of parameters which then adopted by the model for a certain task.

Supervised fine-tuning :

- In this SFT we consider a dataset of qna. On that a set of document is passed with some set of corresponding questions with their answers. So basically it improves the ability of reasoning the answer in a better way.
- Train => Q(questions) --> A(answer) but using RAG interface => D(dataset) + Q(questions) --> A(answer)

So we can see the result over here,

```
Question: The Oberoi family is part of a hotel company that has a head office
in what city?

context: [The Oberoi family is an Indian family that is famous for its
involvement in hotels, namely through The Oberoi Group]...[It is located in
city center of Jakarta, near Mega Kuningan, adjacent to the sister JW Marriott
Hotel. It is operated by The Ritz-Carlton Hotel Company. The complex has two
towers that comprises a hotel and the Airlangga Apartment respectively]...[The
Oberoi Group is a hotel company with its head office in Delhi.]

Instruction: Given the question, context and answer above, provide a logical
reasoning for that answer. Please use the format of: ##Reason: {reason}
##Answer: {answer}.

--------------------------------------------------------------------------------

CoT Answer: ##Reason: The document ##begin_quote## The Oberoi family is an
Indian family that is famous for its involvement in hotels, namely through The
Oberoi Group. ##end_quote## establishes that the Oberoi family is involved in
the Oberoi group, and the document ##begin_quote## The Oberoi Group is a hotel
company with its head office in Delhi. ##end_quote## establishes the head
office of The Oberoi Group. Therefore, the Oberoi family is part of a hotel
company whose head office is in Delhi. ##Answer: Delhi
```

*fig. is from the official research paper -https://arxiv.org/pdf/2403.10131

Let's do some practical implementation..
For Fine-Tuning a model from huggingface follow :

- Setup the google collab..
- Install all the required libraries via " !pip install "
- Set the dataset :
  - 1st of all get the dataset you want to use, either from huggingface or kaggle dataset, I will prefer huggingface dataset for simplicity and easier set up or even you can consider kaggle dataset.
  - I wil use a finance dataset from huggingface as an example.
    - 

  - If you want to use dataset from kaggle then,
    - 

load the datset

```
from google.colab import files
files.upload()
```

Show hidden output

```
import os
import shutil

os.makedirs(os.path.expanduser("~/.kaggle"), exist_ok=True)
shutil.move("kaggle.json", os.path.expanduser("~/.kaggle/kaggle.json"))

# Step 3: Set permissions
os.chmod(os.path.expanduser("~/.kaggle/kaggle.json"), 0o600)
```

Show hidden output

```
import os

os.environ['KAGGLE_USERNAME'] = "you_user_name"
os.environ['KAGGLE_KEY'] = "you_kaggle_key"
```

```
!kaggle datasets list
```

Show hidden output

it downloads a .zip file which we have to unzip it later

```
!kaggle datasets download -d rauffauzanrambe/fifa-world-cup-2026-player-performance-dataset

Dataset URL: https://www.kaggle.com/datasets/rauffauzanrambe/fifa-world-cup-2026-player-performance-dataset
License(s): MIT
Downloading fifa-world-cup-2026-player-performance-dataset.zip to /content
100% 3.96M/3.96M [00:01<00:00, 2.66MB/s]
```

unziped the file inside the content/kaggle/

```
!unzip fifa-world-cup-2026-player-performance-dataset.zip -d ./kaggle

Archive:  fifa-world-cup-2026-player-performance-dataset.zip
  inflating: ./kaggle/fifa_world_cup_2026_player_performance.csv
```

- I have used FIFA world cup dataset, to load a dataset we have to follow this :
  `!kaggle datasets download -d <author_name>/<dataset_name>`

```
import pandas as pd
df = pd.read_csv("./kaggle/fifa_world_cup_2026_player_performance.csv")
df.head(3)
```

	player_id	player_name	age	nationality	team	jersey_number	position	height_cm	weight_kg	preferred_foot	...	possession_impact	pressure_resistan
0	P00055	Rodri Fati	26	Spanish	Spain	3	Goalkeeper	195	75	Left	...	1.1	44
1	P00070	Ansu Le Normand	19	Spanish	Spain	18	Midfielder	178	75	Right	...	3.5	38
2	P00066	Gavi Ramos	18	Spanish	Spain	14	Midfielder	177	72	Left	...	15.3	99

3 rows × 75 columns

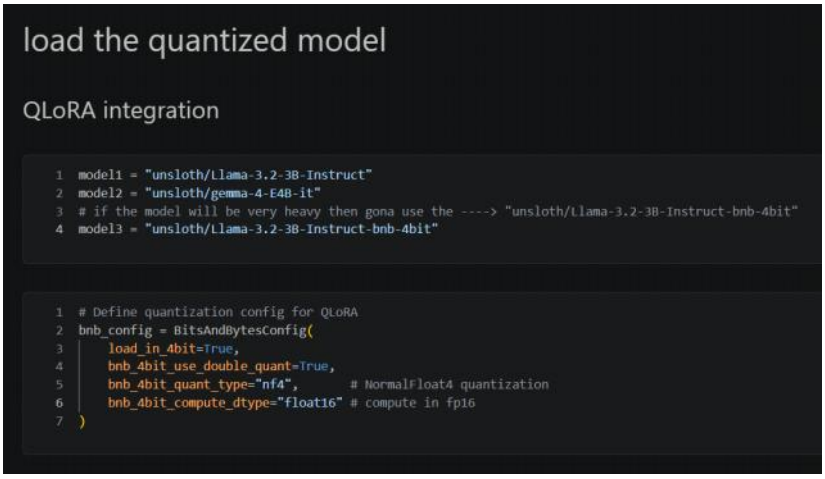
- As you can see that I got the dataset from the kaggle, else if you are doing on local system do the same way else can also directly download it from the website and use it.

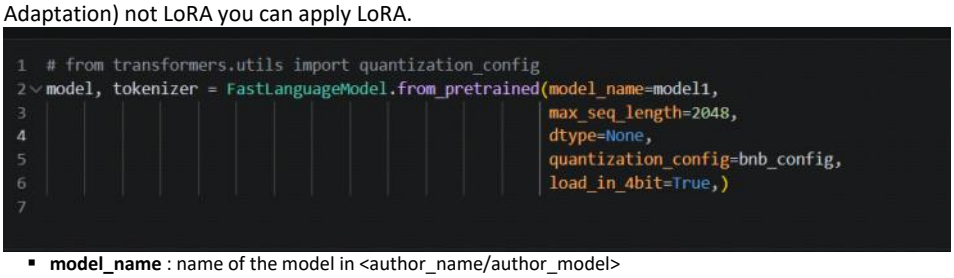
- Now we have to fine-tune the model, we can use unsloth to do that cause unsloth provides an easy process to do fine-tuning by dynamically reducing the GPU uses and offloading the GPU and CPU uses manually to avoid OOM condition and apply LoRA or QLorRA upon the model training, and also can see a drastic change in system use. Like to fine-tuna a gemm4e4b model which is quantized need more than 10gb which get decreases to 5-6gb

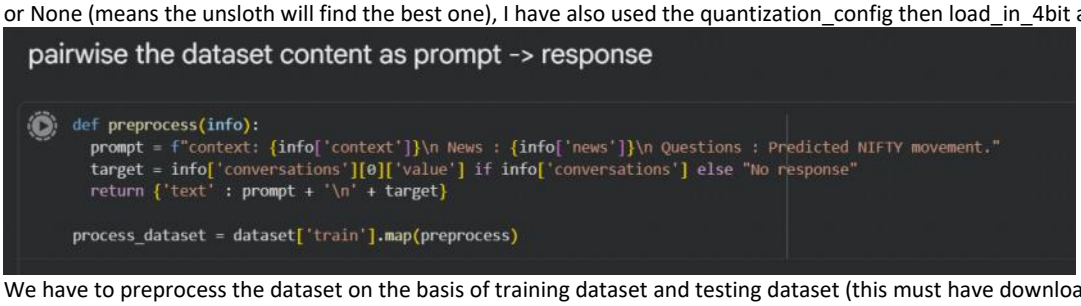
- To run on local system create a virtual environment and install all the dependencies:
  - Install the pytorch for cuda -
    - pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu130
  - Install the unsloth package with all auxiliary dependencies for fine-tuning without overwriting -
    - pip install --no-deps "trl<0.9.0" peft accelerate bitsandbytes
    - pip install "unsloth[conda] @ git+https://github.com/unslothai/unsloth.git"
  - After that make sure all are installed :
    - python -c "import torch; import unsloth; print('CUDA Available:', torch.cuda.is_available()); print('Unsloth Version:', unsloth.__version__)"
- Now you will can do the fine-tuning a base model or a quantized model, make sure that the model is not a GGUF file cause the mathematical computation required were got frozen so we can't apply fine-tune on it.
- Fine-Tunning:
  -

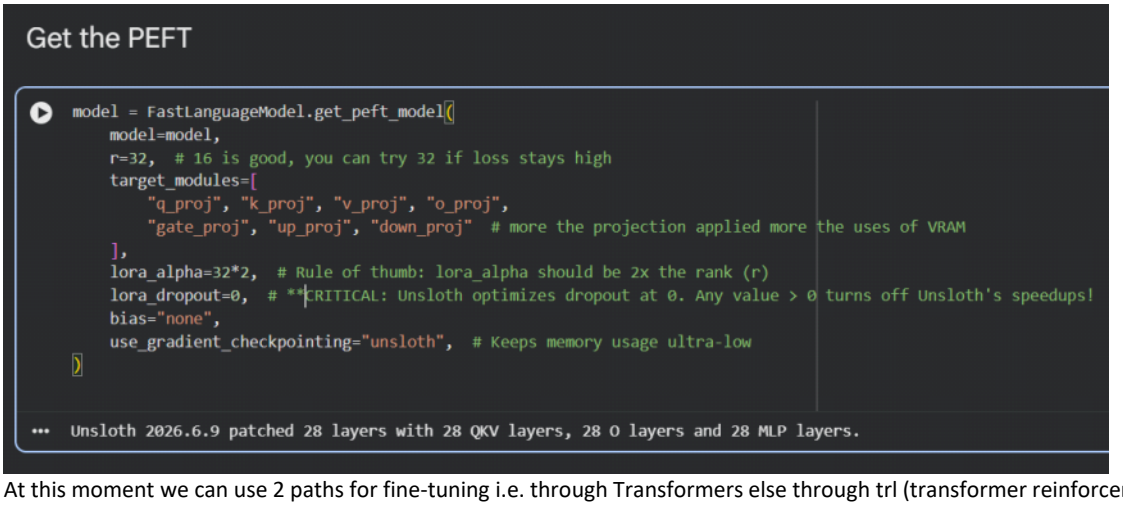
```
from unsloth import FastLanguageModel
from transformers import  AutoTokenizer, TrainingArguments, Trainer, BitsAndBytesConfig
```

```
🦥 Unsloth: Will patch your computer to enable 2x faster free finetuning.
W0626 14:09:10.020000 14680 site-packages\torch\utils\_pytree.py:630] <enum 'KernelPreference'> is an Enum subcl
W0626 14:09:10.113000 14680 site-packages\torch\utils\_pytree.py:630] <enum 'ScaleCalculationMode'> is an Enum s
🦥 Unsloth Zoo will now patch everything to make training faster!
<string>:1: FutureWarning: torch._dynamo.config.inline_inbuilt_nn_modules is deprecated and does not do anything
```

- 

  - **Load_in_4bit** : tell the loader to store the model weight in 4 bit, it allow to cut the memory uses where a large model can easily fit in small GPU.
  - **bnb_4bit_use_double_quant** : enables double quantization (2 times quantization), further reduces memory footprint while keeping accuracy higher than plain 4bit quantization.
  - **bnb_4bit_quant_type** : choose the quantization schema, 'nf4' gives more better accuracy than the basic int4 quantization.
  - bnb_4bit _compute_dtype : sets the precision used during computation (matrix multiplication and forward passes), even though weights are stored in 4bit but they are cast to FP16 during compute this balances speed and accuracy- faster than FP32.
- You can pre-define the model(s) you are going to fine-tune. I have used a bnb_config which means I'm applying QLoRA(quantized Low Rank Adaptation) not LoRA you can apply LoRA.
- 

  - **model_name** : name of the model in <author_name/author_model>
  - **max_seq_length** : maximum length each time the model will be trained
  - **dtype** : defines the datatype for model weight and computation, 'None' means that unsloth will decide which to choose.
  - **quantization_config** : BitsandBytes quantization, ensures the model loads with the chosen quantization scheme.
  - **load_in_4bit** : tell the loader to store the model weight in 4 bit, it allow to cut the memory uses where a large model can easily fit in small GPU.
- Using **unlsoth** we can separate out the model weight and the tokenizer using from_pretrained from FastLanguageModel. Here the model_name is the repo and the model we are using from the huggingface (all the models we going to use must be from the huggingface) and the max_seq_length means the maximum length of the sentence per training, and the dtype means whether to use float16 or bfloat16 or None (means the unsloth will find the best one), I have also used the quantization_config then load_in_4bit as True simply a Boolean.
- 

- We have to preprocess the dataset on the basis of training dataset and testing dataset (this must have downloaded while downloading from huggingface dataset) according to your dataset.
- 

- At this moment we can use 2 paths for fine-tuning i.e. through Transformers else through trl (transformer reinforcement learning), At the end both are the same thing while the trl is more advanced and widely used for fine-tuning as it has RLHF/RLAIF means Reinforcement

Learning with Human/AI Feedback. And It has Reward modeling (RM) means train a 2ndry model that understand what constitutes a good or bad response based on human or automated feedback.

- The key trainers are :
  - SFTTrainer : supervised fine-tuning
  - RewardTrainer : to create your own reward criteria
  - PPOTrainer : to optimize with RLHF
  - DPOTrainer : for preference based alignment

- If we use direct via Transformers then..
  - set up for training

```
from transformers import TrainingArguments
import torch

training_args = TrainingArguments(
    output_dir="./gemma4",
    per_device_train_batch_size=3,
    gradient_accumulation_steps=4,
    num_train_epochs=3,
    learning_rate=2e-4,
    lr_scheduler_type="cosine",
    warmup_ratio=0.03,
    logging_steps=10,
    save_steps=200,
    save_total_limit=2,
    optim="adamw_8bit",
    weight_decay=0.01, # Prevents overfitting during the cosine decay phase
    fp16=not torch.cuda.is_bf16_supported(),
    bf16=torch.cuda.is_bf16_supported(),  # Uses modern 16-bit precision to prevent gradient underflow
)
```

```
warmup_ratio is deprecated and will be removed in v5.2. Use `warmup_steps` instead.
```

  - Output_dir : where the checkpoints and logs will be saved at, to keep things organized and recoverable.
  - Per_device_train_batch_size : number of samples per GPU per steps, it helps to manage the memory usage means less the number less stress on GPU
  - Gradient_accumulation_steps : accumulates gradients over 4 steps before the weights are updated.
  - Num_train_epochs : number of full passes through the dataset, it defines the duration of the training but more epochs tends to overfitting risk.
  - Learning_rate : step size for weight updates, more the value -> unstable, less the value -> slow means it balances the speed and stability of training.
  - Lr_scheduler_type : smoothly reduces the learning rate, often improving the convergence, the learning rate scheduler that decays the following in a cosine curve.
  - Warmup_ratio : fraction of steps that ramps up the initial learning rate.
  - Logging_steps : frequency of logging, helps in monitoring the progress.
  - Save_steps : saves checkpoint after every 200 steps, it helps in recovery and evaluation during training.
  - Save_total_limilt : save only the latest n steps (save latest 2 steps), helps in managing storage/memory by deleting previous.
  - Optim : types of optimizer like adamw_8bit (AdamW, SGD, Adafactor, LAMB, Lion) use to reduce the memory uses while maintaining the accuracy.
  - Weight_dacay : penalizes the large weights, helps prevent from overfitting especially during cosine decay.
  - Fp16 : enables fp16 if the BF16 is not supported, saves memory and speed up training on GPU that doesn't support BF16.
  - Bf16 : enable bf16 if supported.

```
trainer = Trainer(model=model,
                  args=training_args,
                  train_dataset= tokenized_dataset,)

trainer.train()
```

```
==((====))==  Unsloth - 2x faster free finetuning | Num GPUs used = 1
   \\   /|    Num examples = 1,477 | Num Epochs = 3 | Total steps = 372
O^O/ \_/ \    Batch size per device = 3 | Gradient accumulation steps = 4
\        /    Data Parallel GPUs = 1 | Total batch size (3 x 4 x 1) = 12
 "-____-"     Trainable parameters = 48,627,712 of 3,261,377,536 (1.49% trained)
`use_return_dict` is deprecated! Use `return_dict` instead!
Unsloth: Will smartly offload gradients to save VRAM!
Unsloth: Double buffering enabled (parallel H2D + compute) for backward pass.
[229/372 30:28 < 19:12, 0.12 it/s, Epoch 1.84/3]
```

Step	Training Loss
10	3.429427
20	3.207913
30	3.163198
40	3.140020

- Else if you want to train using SFTTrainer then:

```
from trl import SFTTrainer, SFTConfig
trl_args = SFTConfig(
        per_device_train_batch_size=3,
        gradient_accumulation_steps=4,
        warmup_steps=10,
        warmup_ratio=0.05,
        learning_rate=2e-4,
        lr_scheduler_type="cosine",
        num_train_epochs=4, # Using this value
        max_seq_length=2048,
        packing=True,
        # max_steps=60,
        logging_steps=1,
        output_dir="llama3.2",
        optim='adamw_8bit',
    )
```

```
warmup_ratio is deprecated and will be removed in v5.2. Use `warmup_steps` instead.
```

- Output_dir : where the checkpoints and logs will be saved at, to keep things organized and recoverable.
- Per_device_train_batch_size : number of samples per GPU per steps, it helps to manage the memory usage means less the number less stress on GPU
- Gradient_accumulation_steps : accumulates gradients over 4 steps before the weights are updated.
- Num_train_epochs : number of full passes through the dataset, it defines the duration of the training but more epochs tends to overfitting risk.
- Learning_rate : step size for weight updates, more the value -> unstable, less the value -> slow means it balances the speed and stability of training.
- Lr_scheduler_type : smoothly reduces the learning rate, often improving the convergence, the learning rate scheduler that decays the following in a cosine curve.
- Warmup_ratio : fraction of steps that ramps up the initial learning rate.
- Logging_steps : frequency of logging, helps in monitoring the progress.
- Warmup_step : number of training steps should be used for warmup, independent of total training length.

```
def formatting_func(example):
    # The example['conversations'] is already a list of dictionaries
    # We apply the tokenizer's chat template to format it into a single string.
    # Set add_special_tokens=False as SFTTrainer often adds them internally.
    text = tokenizer.apply_chat_template(example['conversations'], tokenize=False, add_special_tokens=False)
    # SFTTrainer expects a list of processed strings from formatting_func
    return [text]

trainer = SFTTrainer(
    model=model,
    tokenizer = tokenizer,
    train_dataset=process_dataset, # process_dataset now contains only the 'conversations' column
    max_seq_length=2048,
    args=trl_args,
    formatting_func=formatting_func, # Provide the formatting function
    # packing=True # Ensure this is uncommented if you intend to use packing for efficiency
)
```

```
Unsloth: Tokenizing ["text"] (num_proc=6): 100% 1477/1477 [00:42<00:00, 72.39 examples/s]
Unsloth: Packing train dataset (num_proc=6): 100% 1477/1477 [00:00<00:00, 592.38 examples/s]
Unsloth: Packing enabled - training is >2x faster and uses less VRAM!
```

```
trainer.train()
```

```
==((====))==  Unsloth - 2x faster free finetuning | Num GPUs used = 1
   \\   /|    Num examples = 1,466 | Num Epochs = 1 | Total steps = 60
O^O/ \_/ \    Batch size per device = 3 | Gradient accumulation steps = 4
\        /    Data Parallel GPUs = 1 | Total batch size (3 x 4 x 1) = 12
 "-____-"     Trainable parameters = 97,255,424 of 3,310,005,248 (2.94% trained)
`use_return_dict` is deprecated! Use `return_dict` instead!
Unsloth: Will smartly offload gradients to save VRAM!
Unsloth: Double buffering enabled (parallel H2D + compute) for backward pass.
[13/60 05:57 < 25:27, 0.03 it/s, Epoch 0.10/1]
```

Step	Training Loss
1	2.335275
2	2.436807

- After that you will find this kind of training loss and steps, at the end we will get the lowest loss. If the loss is still high we can change the training epochs and batch size, gradient accumulation step change and play with the parameters to get the best lowest training loss.
- After the training completes we will test it, how we get the response after training out model:
  - We can directly test using python and then if the training loss is low and we get the response as we needed we will register the gguf file to the Ollama and use on projects.

```
FastLanguageModel.for_inference(model)
messages = [{
    "role": "user",
    "content": "stock trend of ADANIPOWER?"
}]
inputs = tokenizer.apply_chat_template(messages, tokenize=True, add_generation_prompt=True, return_tensors='pt').to('cuda')
outputs = model.generate(input_ids=inputs, max_new_tokens=2048, use_cache=True, temperature=0.05, do_sample=True,top_p=0.9)
response = tokenizer.batch_decode(outputs)[0]
print(response)
```

  - Here we will get the refined LLM generated response which might contain extra noises, cause it will be generated by our LLM.

-
```
import json

# Extract a user message directly from the training dataset example
# We'll use dataset["train"][1] as it was printed in a previous cell.
example_conversation = dataset["train"][1]['conversations'][0]

# The content for the user's message should be the 'value' from the conversation.
# Ensure it's a list of dictionaries as expected by apply_chat_template.
messages_for_inference = [{
    "role": example_conversation['role'],
    "content": example_conversation['value']
}]

print(f"\n--- User Query from Training Data ---\n{messages_for_inference[0]['content']}\n---\n")

inputs = tokenizer.apply_chat_template(messages_for_inference, tokenize=True, add_generation_prompt=True, return_tensors='pt').to('cuda')
outputs = model.generate(input_ids=inputs, max_new_tokens=2048, use_cache=True, temperature=0.05, do_sample=True, top_p=0.9)
response = tokenizer.batch_decode(outputs)[0]
print(f"\n--- Model Generated Response ---\n{response}\n---\n")

# We can also compare to the expected target from the dataset example
expected_response = dataset["train"][1]['conversations'][1]['value']
print(f"\n--- Expected Response from Dataset ---\n{expected_response}\n---\n")
```

-
```
2010-01-05,113.26,113.68,112.85,113.63,87.7148,111579900.0,0.0026,0.8299,113.771,109.
2010-01-06,113.52,113.99,113.43,113.71,87.7765,116074400.0,0.0007,0.8848,114.0852,109
...

Britain Set for Next Step on Wind Power
Journal Register Taps Paton as CEO
Crude Settles

        Below $83
Gold Pressured by Dollar
European Stocks Trade in Tight Ranges
Shanghai Slides on PBOC Move
Reliance MediaWorks Buys Film Processing Firm ilab UK
Exchange Tightens ChiNext Rules
Prudential to Buy UOB Unit
Dollar, Commodities Break Usual Tie
Small Banks Say a Cure Hurts
Music Sales Sink Further
Mesa Air Files Chapter 11
Dollar Slide Deepens After Fed Minutes
U.S. Airlines Claw Back
Symetra Gets 2010 IPO Market Rolling
Aberdeen Leads Race Over RBS Unit
Judge Ends Lawsuit Against Wells Fargo
U.K.'s Gas Buffers Struggle With Cold
U.S. Now a Renters' Market
FDIC Weighs Tying Fees to Banks' Pay
Answer:<|eot_id|><|start_header_id|>assistant<|end_header_id|>

Fall, -0.02%<|eot_id|>
---


--- Expected Response from Dataset ---
Neutral
0.0033
```

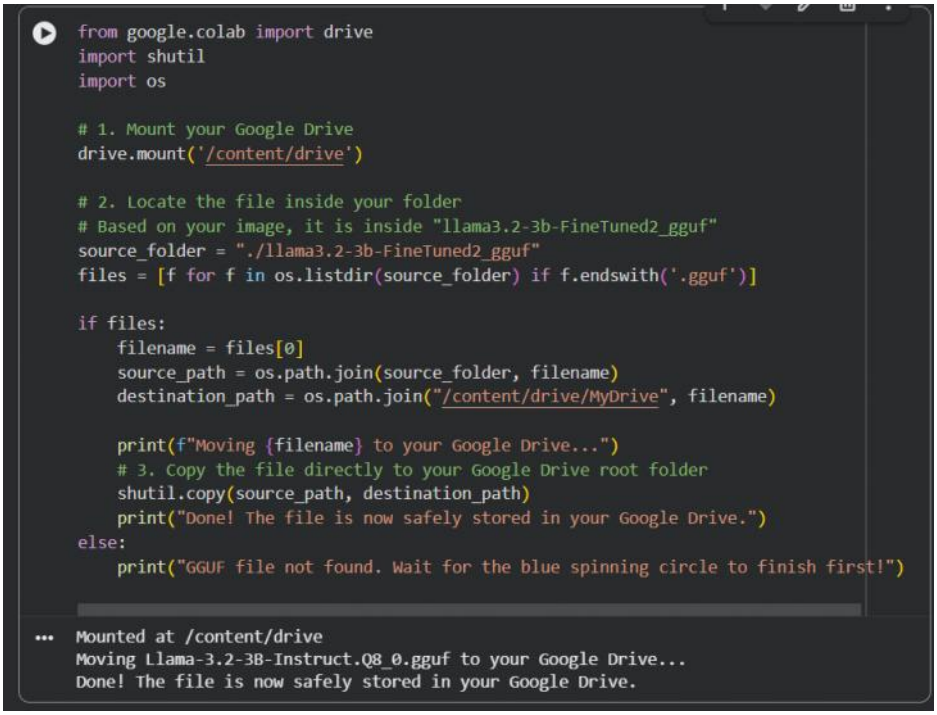
  - I got this kind of answer, not correct but still managed to response, I have to change the dataset quality and the SFT parameters too. *This was for example
  - This will provide the raw context generated before passing to the LLM model this helps to compare how the raw response got generated with the real output.

So now we have completed our fine-tuning completely, then we will install the .gguf file of the fine-tuned model:

-
else if you take the direct gguf file to register to ollama and play then..

```
from google.colab import files
files.download('./llama3.2:3b-FineTuned2.gguf')
# To download the tokenizer, you would typically zip the directory first or download its contents.
# files.download('./llama3.2:3b-FineTuned_tokenizer') # This will not work as it's a directory
```

  - Else directly connect to your google drive and then push into it. *priority if you are doing on google collab

- 

```
from google.colab import drive
import shutil
import os

# 1. Mount your Google Drive
drive.mount('/content/drive')

# 2. Locate the file inside your folder
# Based on your image, it is inside "llama3.2-3b-FineTuned2_gguf"
source_folder = "./llama3.2-3b-FineTuned2_gguf"
files = [f for f in os.listdir(source_folder) if f.endswith('.gguf')]

if files:
    filename = files[0]
    source_path = os.path.join(source_folder, filename)
    destination_path = os.path.join("/content/drive/MyDrive", filename)

    print(f"Moving {filename} to your Google Drive...")
    # 3. Copy the file directly to your Google Drive root folder
    shutil.copy(source_path, destination_path)
    print("Done! The file is now safely stored in your Google Drive.")
else:
    print("GGUF file not found. Wait for the blue spinning circle to finish first!")
```

```
Mounted at /content/drive
Moving Llama-3.2-3B-Instruct.Q8_0.gguf to your Google Drive...
Done! The file is now safely stored in your Google Drive.
```

We can now implement our simple RAG system into it so that our domain specific agent can now generate more accurate than a base model with the RAG system.
For that you can refer my "RAG system overview" document.

Now we can use our own fine-tuned model on various use cases or also can make a group of worker agents under a specific evaluator agent and we will do this on upcoming topic document.

Thanks for reading this document, feel free to contact if something is wrongly or not properly described.
@metamorphtech.uk@gmail.com
GitHub : https://github.com/swayansu951